

Übung: Datenbankmodellierung (NoSQL)

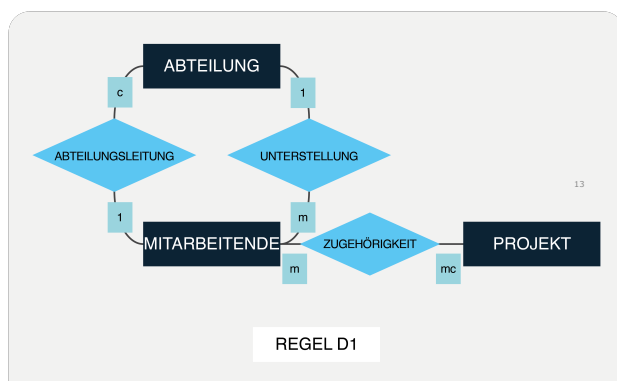
1. Lernziele

- Die Datenstruktur von Quelldaten analysieren
- Ein Datenmodell für Dokumentendatenbanken entwerfen mit einem Entity Relationship Document Diagram (ERDD)
- Ein grundlegendes Verständnis der JSON-Datenstruktur
- Dokumenttypen mit der JSON-Prototyp-Methode modellieren

2. Buch SQL & NoSQL Datenbanken

- Lesen Sie Kapitel 2.4 – 2.5 im Buch von Kaufmann & Meier (2023). Sie finden das eBook der 9. Auflage auf ILIAS.
- ? Was sind Unterschiede und Gemeinsamkeiten bei der Abbildung von ER-Diagrammen auf Graph- bzw. Dokument-Strukturen?
- ? Auf welche Arten kann die Modellierung von Dokumentstrukturen das Big Data Management unterstützen?
- ? Wie werden Entitäten in Dokumenten modelliert?

Abbildung von ausgewählten Entitätsmengen und Attributen auf Objekte und Eigenschaften

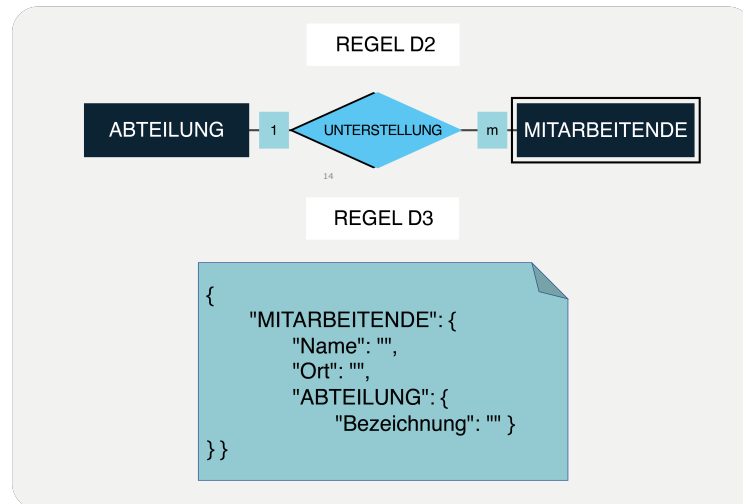


HSLU

© Springer 2023, Kaufmann M., Meier A.: SQL & NoSQL-Databases

- ? Wie werden 1:1, 1:n und n:m-Beziehungen in Dokumenten modelliert?

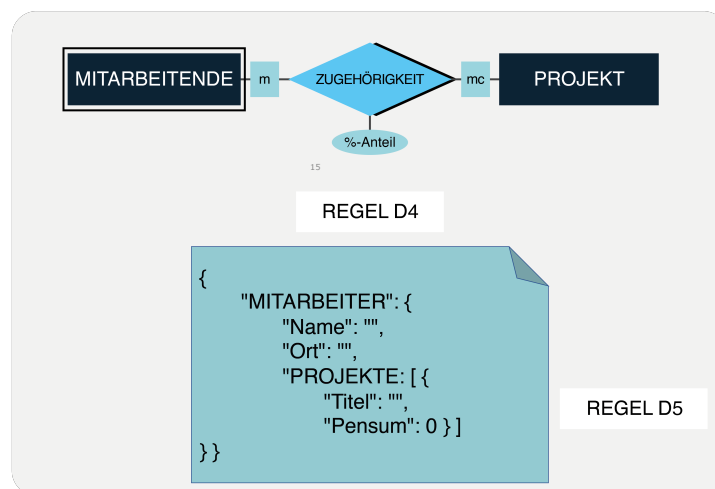
Aggregation einer eindeutigen Assoziation als Feld mit Unterobjekt



HSLU

© Springer 2023, Kaufmann M., Meier A.: SQL & NoSQL-Databases

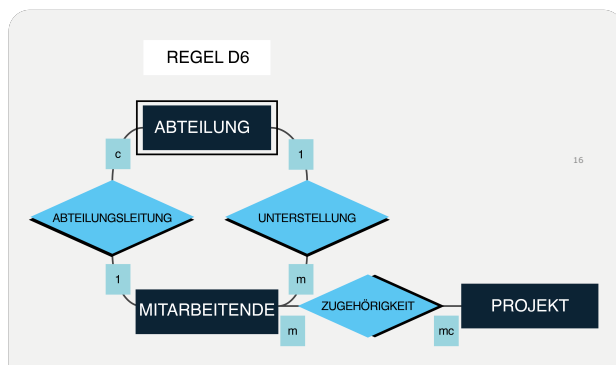
Abb. 2.28: Aggregation einer mehrdeutigen Assoziation als Feld mit Liste von Unterobjekten, inklusive Beziehungsattributen



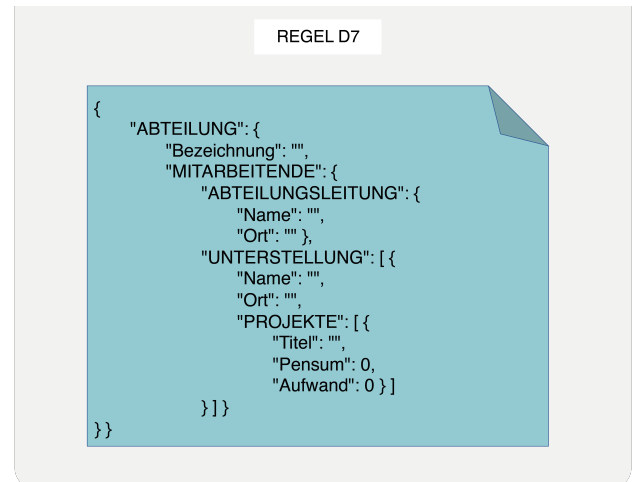
HSLU

? Wie gehen wir mit mehreren Beziehungen zwischen gleichen Entitäten in Dokumenten um?

Abb. 2.29: Dokumenttyp ABTEILUNG mit Aggregation der gleichen Entitätsmenge MITARBEITENDE in zwei unterschiedlichen Assoziationen ABTEILUNGSLEITUNG und UNTERSTELLUNG



HSLU



3. Wissenschaftliche Literatur (optional)

- Studieren Sie den Text von Parsons und Wand über die Datenbankmodellierung ohne inhärente Klassifikation: Parsons and Y. Wand, "Emancipating Instances from the Tyranny of Classes in Information Modeling," *ACM Trans. Database Syst.*, vol. 25, no. 2, pp. 228–268, Jun. 2000, doi: [10.1145/357775.357778](https://doi.org/10.1145/357775.357778).

? Was heisst inhärente Klassifikation im Datenbank-Design?

In klassischen Datenbankmodellen (z. B. relational,) wird jeder Datensatz einer **festen Tabelle, d.h.. einer Klassifikation** zugeordnet.

Inhärent bedeutet: die Zugehörigkeit zu einer Klasse ist für einen Datensatz **essentiell und unveränderlich**.

Die Notwendigkeit, Datensätze immer einer Klasse zuzuordnen zu müssen

? Zu welchen Problemen kann inhärente Klassifikation führen, und warum?

Dynamische Realitäten schlecht abbildbar: In der Realität können sich Dinge ändern (z. B. ein „Student“ wird später ein „Mitarbeiter“). In DB-Modellen mit starrer Klassifikation muss man Objekte migrieren, löschen oder neu erzeugen.

Semantische Einschränkungen: Ein Objekt kann oft **mehrere Rollen** einnehmen (z. B. eine Person kann gleichzeitig „Kunde“ und „Lieferant“ sein). Feste Klassenzugehörigkeit schränkt die Modellierungsmöglichkeiten ein.

Evolution und Erweiterung: Wenn sich Anforderungen ändern, sind Datenmodelle mit fester Klassifikation oft **nicht flexibel** genug.

Mehrfache Klassifikation ist unklar: Integration verschiedener Benutzersichten ist komplex. Integration verschiedener Schemas in unterschiedlichen Systemen extrem aufwändig, wenn alle Klassen erst vereinheitlicht werden müssen

Änderungen am Schema sind bei fixer Klassifikation aufwändig: Unklar, wie mit neuen, bisher nicht vorgesehenen Änderungen umgegangen werden soll Löschanomalien, wenn Klassen (Tabellen) entfernt werden sollen

? Was ist aus Sicht der Autoren die Alternative?

Instance models and independent class models => class independence

? Was bedeutet class independence?

Unabhängigkeit der Datensätze von der Klasse in denen sie eingeordnet werden

? Inwiefern entspricht dies dem Ansatz zum Datenbank-Design, wie wir ihn in heutigen Dokument-Datenbanksystemen kennen?

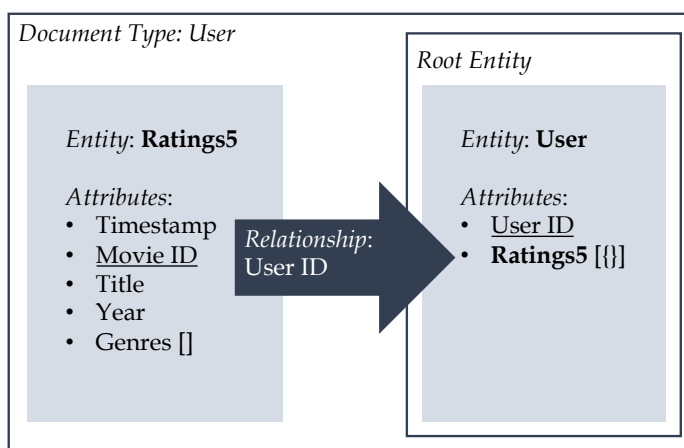
Two layered => documents and collections

Instances: JSON. Collections: reine Sammlungen von Dokumenten, keine Klassen.

Class independence: Dokumente werden nicht durch Collections klassifiziert => schema freedom, beliebige Struktur in jeder Collection möglich

4. MongoDB Arbeitsbuch Kapitel 3

- Auf ILIAS finden Sie das dritte Kapitel des MongoDB-Arbeitsbuchs, das Ihnen Schritt für Schritt bei der Erstellung einer NOSQL-Datenbankanwendung helfen wird.
 - Arbeiten Sie Kapitel 3 durch: "Define the Database Structure".
 - Führen Sie die Übung am Ende des Kapitels 3 durch:
 - Modellieren Sie ein JSON-Dokumenttyp, in dem Sie die Filmdetails in einem Dokument zusammenfassen, das die Informationen zu Benutzer, Bewertung und Film zusammen enthält.
 - Zeichnen Sie zunächst ein Entity Relationship Document Diagram (ERDD).
 - Schreiben Sie dann einen JSON-Prototyp, um die Dokumentstruktur zu definieren.



```
{
  "user": {
    "userId": 0,
    "ratings5": [
      {
        "movieId": 0,
        "timestamp": "",
        "Title": "",
        "Year": 0,
        "Genres": ["" ]
      }
    ]
  }
}
```

5. Semesterprojekt

- Für Ihr Semesterprojekt können Sie in dieser Woche die Datenbankstruktur der Dokumentdatenbank definieren.
- Wie sieht das Entity-Relationship-Document-Diagramm (ERDD) aus?
- Welche Änderungen ergeben sich zum ER-Diagramm der Quelldaten?
- Welche Performance-Überlegungen stehen dahinter?
- Wie sehen die JSON-Prototypen der Ziel-Datenbankstruktur aus?
- Stellen Sie bitte den aktuellen Stand Ihrer Projektarbeit bis und mit der Datenmodellierung dar und geben Sie einen Einblick in Ihren Projektbericht.

6. Projektpräsentation

- Ein Team wird in der nächsten Sitzung den Status ihres Projekts bis und mit dem Thema Datenbankmodellierung vorstellen.